

MiniB2B B2B E-TİCARET YÖNETİM SİSTEMİ

TEKNİK MİMARİ VE SİSTEM TASARIM DOKÜMANI

(N-Tier, ER, UML & Akış Diyagramları)

Hazırlayan:

Şevval Öveyik

Yazılım Mühendisi

Teknoloji Yığını: .NET 8 • ASP.NET Core MVC • EF Core • MS SQL

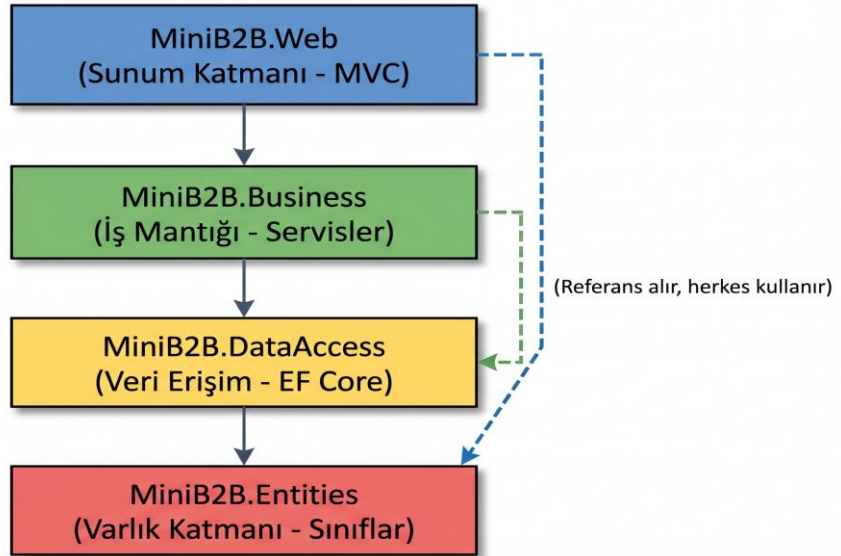
Server Tarih: 2026

MiniB2B Projesi Teknik Mimari ve Tasarım Dokümantasyonu

1. Katmanlı Mimari Şeması (N-Tier Architecture)

Projede kod karmaşasını önlemek, iş kuralları ile arayüzü birbirinden ayırmak için 4 ana katmandan oluşan N-Tier mimarisi tercih edildi.

- **MiniB2B.Web (UI):** ASP.NET Core MVC yapısının kurulduğu katman. Sadece kullanıcıdan gelen HTTP isteklerini karşılar, View ve Controller yönetimini yapar. Veritabanına kesinlikle doğrudan erişmez, araya Business katmanını koyar.
- **MiniB2B.Business (İş Mantığı):** Sistemin beyni. Stok yeterli mi, sepet kurallara uygun mu gibi tüm iş kuralları ve validasyonlar buradaki servislerde (OrderService, ProductService) yazıldı.
- **MiniB2B.DataAccess (Veri Erişimi):** Entity Framework Core ve DbContext yapılarının bulunduğu katman. Veritabanı sorguları (CRUD) burada yürütülür.
- **MiniB2B.Entities (Modeller):** Veritabanı tablolarına karşılık gelen saf C# sınıflarıdır (POCO). Hiçbir katmana bağımlı değildir; Web, Business ve DataAccess katmanları ortak modelleri kullanabilmek için bu projeyi referans alır.

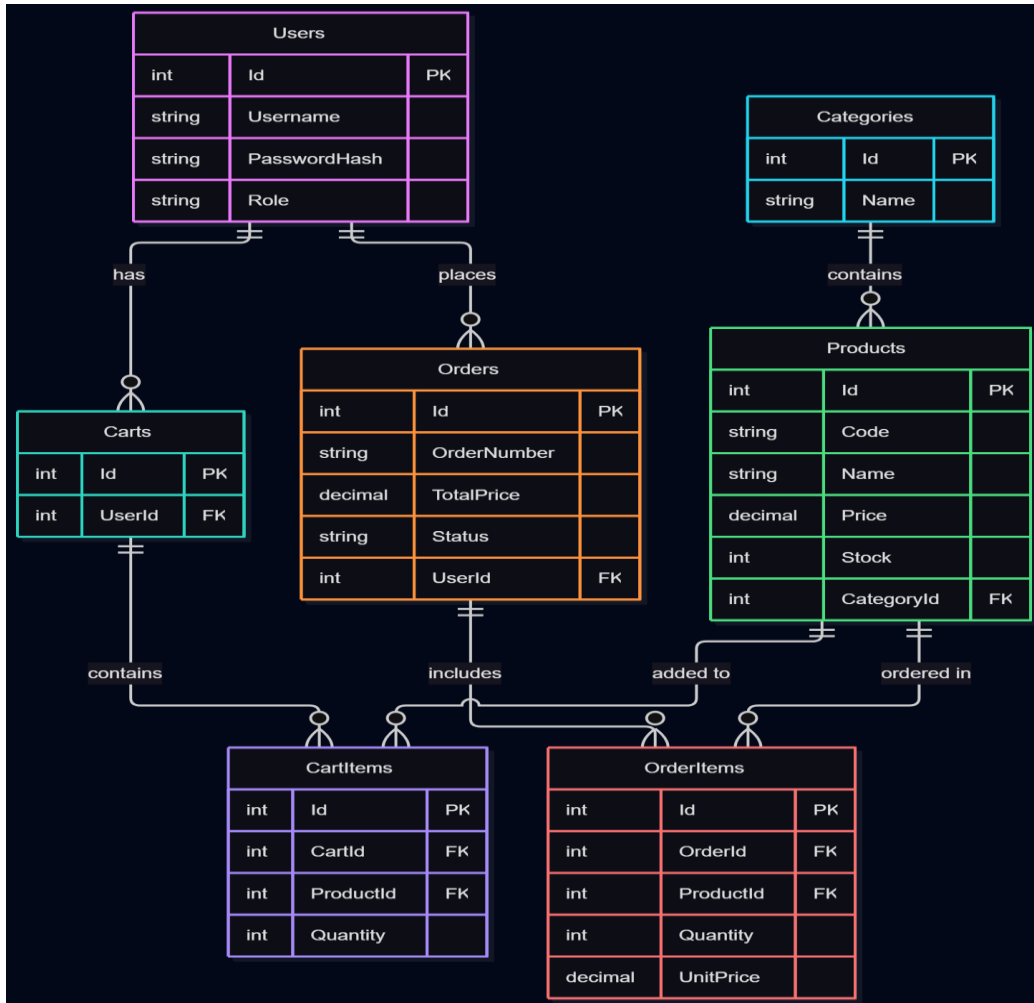


Şekil 1. N-Tier Mimari Şeması

2. Veritabanı ER Diyagramı

İlişkisel veritabanı yapısı Entity Framework Core Code-First yaklaşımına uygun olarak modellendi.

- **Users:** Sistemdeki tüm kullanıcıları ve rollerini (Customer veya Admin) tutar. Bir kullanıcının birden fazla siparişi ve sepet kaydı olabileceği için **Orders** ve **Carts** ile 1-N ilişki kuruldu.
- **Categories & Products:** Ürünler bir kategoriye bağlıdır. **Products** tablosu ürünün güncel fiyat ve stok durumunu tutan ana tablodur.
- **Carts & CartItems:** Kullanıcıların anlık sepetini temsil eder. **CartItems**, sepet ile ürün arasındaki çoktan-çoğa ilişkiyi çözen ara tablodur ve sepetteki ürün adet bilgisini taşır.
- **Orders & OrderItems:** Tamamlanmış siparişleri saklar. Burada en kritik tasarım: Ürünün o anki satış fiyatı doğrudan **OrderItems** tablosundaki **UnitPrice** kolonuna yazılır. Böylece gelecekte ürünün fiyatı değişse bile geçmiş sipariş faturaları ve toplam tutarlar bozulmaz.

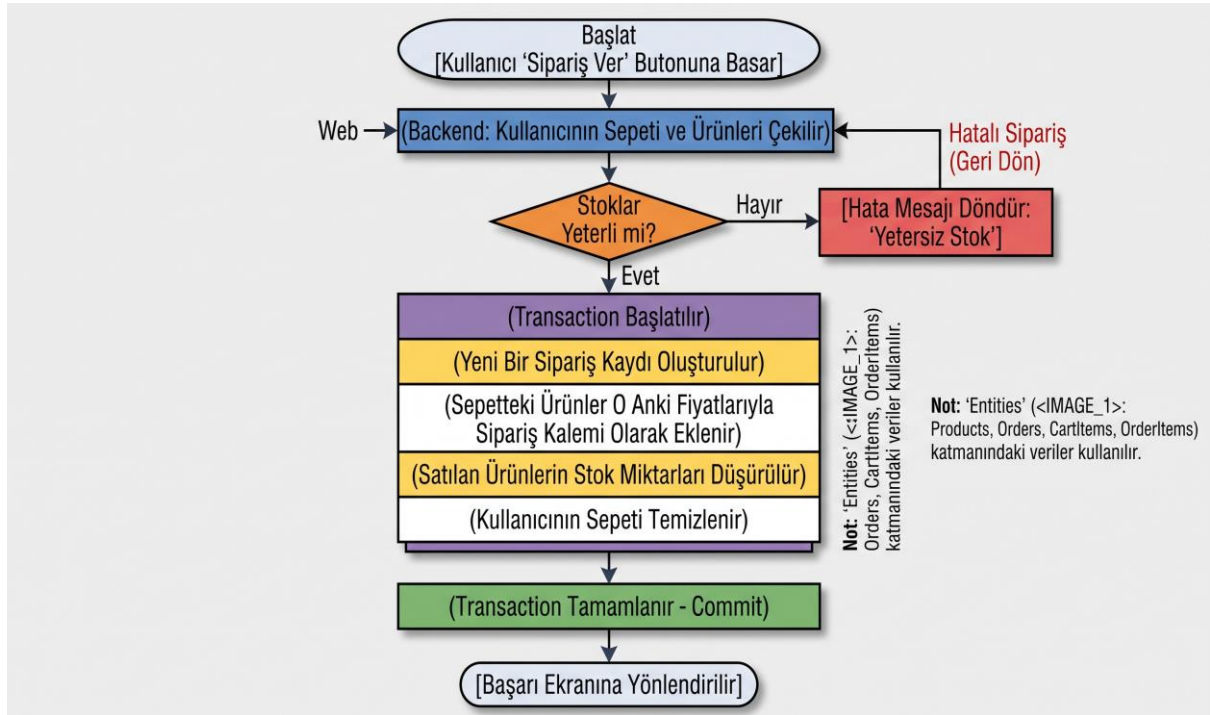


Şekil 2. Veritabanı ER (Varlık-İlişki) Diyagramı

3. Sipariş Oluşturma Akış Şeması (Flowchart)

Sipariş süreci, projenin en kritik iş mantığıdır. Stok tutarsızlıklarını önlemek için adım adım kurgulandı:

1. **Ön Kontrol:** Kullanıcı "Sipariş Ver" dediğinde, Business katmanı sepetteki tüm ürünlerin veritabanındaki güncel stoklarını kontrol eder.
2. **Hata Yakalama:** Tek bir üründe bile stok yetersizse işlem anında kesilir, kullanıcıya net bir hata mesajı dönülür ve sepet ekranına geri yönlendirilir.
3. **Transaction Güvencesi:** Stoklar tamsa işlem bir Database Transaction içine alınır.
4. **Kayıt ve Güncelleme:** Sırayla Orders kaydı açılır, ürünler o anki fiyatlarıyla OrderItems'a yazılır, ana tablodaki stok miktarları düşülür ve kullanıcının sepeti temizlenir.
5. **Commit:** Tüm bu işlemler hatasız biterse Transaction Commit edilir. Eğer aradaki herhangi bir adımda (örneğin stok düşerken) hata çıkarsa işlem Rollback edilir; böylece sipariş kaydı açılıp stoğun düşmeme riski tamamen sıfırlanır.

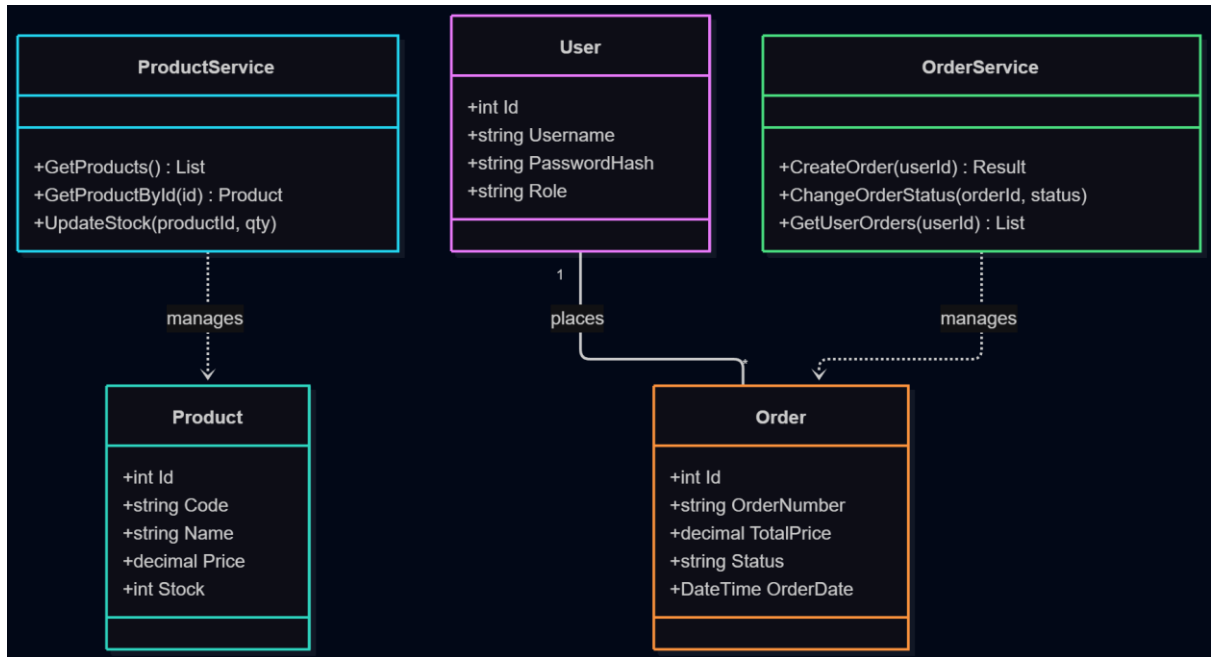


Şekil 3. Sipariş Oluşturma Akış Şeması (Flowchart)

4. Sınıf Diyagramı (Class Diagram)

OOP prensiplerine ve Tek Sorumluluk İlkesine (Single Responsibility) göre sınıflar iki temel grupta toplandı:

- **Entity Sınıfları (User, Product, Order):** Yalnızca veri tutan, property içeren sınıflardır. İçlerinde herhangi bir iş mantığı veya metot barındırmazlar.
- **Servis Sınıfları (ProductService, OrderService):** Veriler üzerinde operasyon yapan motorlardır. Örneğin ProductService stok güncelleme ve ürün arama işlerini yönetirken, OrderService sipariş açma ve sipariş durumunu değiştirme (Pending, Approved vb.) süreçlerini yürütür.

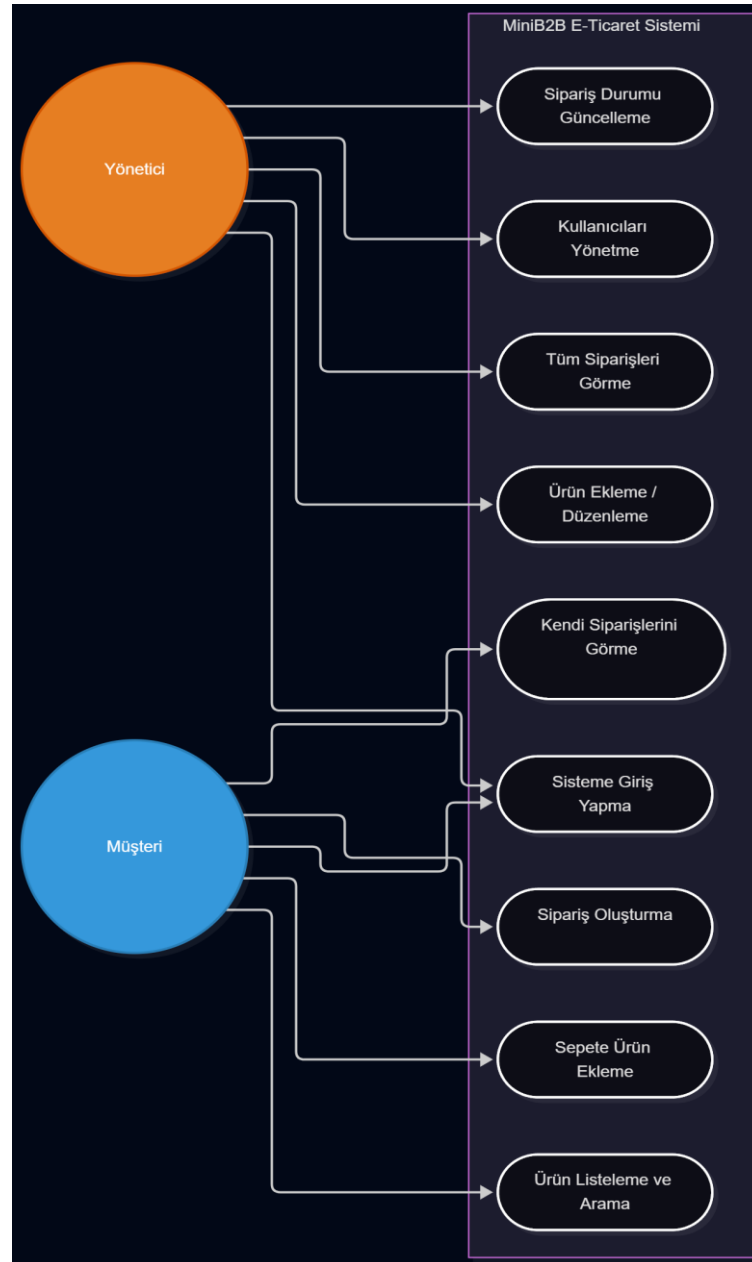


Şekil 4. Sınıf Diyagramı (Class Diagram)

5. Kullanım Senaryosu Diyagramı (Use Case)

Sistemdeki kullanıcı yetkilerini ayırtırmak için iki temel aktör tanımlandı:

- **Müşteri (Customer):** Sisteme giriş yaptıktan sonra ürünleri filtreleyebilir, arayabilir, sepetine ürün ekleyip siparişi tamamlayabilir. Güvenlik gereği sadece kendi geçmiş siparişlerini görebilir.
- **Yönetici (Admin):** Müşteri fonksiyonlarının ötesinde yönetimsel yetkilere sahiptir. Yeni ürün tanımlayabilir, mevcut ürünleri güncelleyebilir, sistemdeki tüm siparişleri görüp siparişin durumunu (Onaylandı, Kargolandı) değiştirebilir ve kullanıcı hesaplarını listeleyebilir

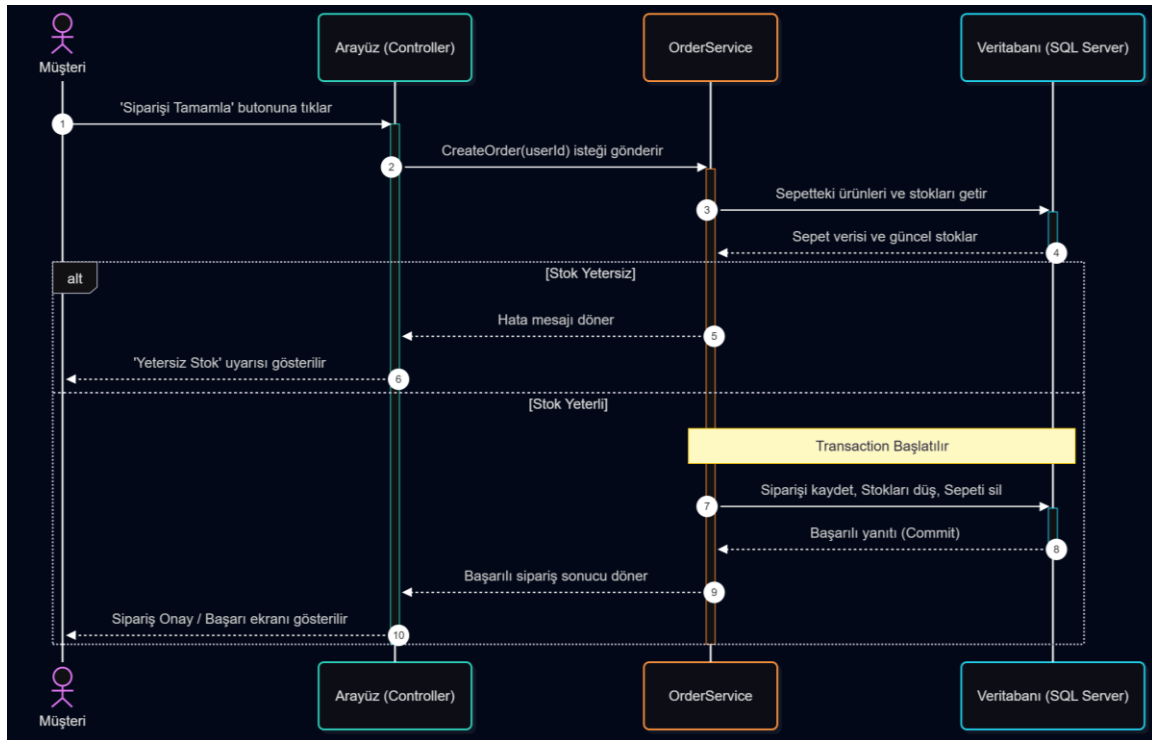


Şekil 5. Kullanım Senaryosu Diyagramı (Use Case)

6. Sipariş Süreci Ardışıklık Diyagramı (Sequence Diagram)

Sipariş onayının kod tarafında hangi sırayla ve hangi katmanlar üzerinden aktığını gösterir:

1. Kullanıcı arayüzde (Web) butona tıklar.
2. OrderController, gelen isteği OrderService.CreateOrder(userId) metoduna iletir.
3. Servis, SQL Server'a gidip sepeti ve anlık ürün stoklarını okur.
4. Stok yetersizse akış burada biter, Controller kullanıcıya hata gösterir.
5. Stok yeterliyse Transaction başlatılır; sipariş yazılır, stoklar düşülür ve sepet temizlenir.
6. SQL Server başarılı yanıtı döndüğünde işlem commit edilir, Controller kullanıcıyı "Siparişiniz Alındı" ekranına yönlendirir.

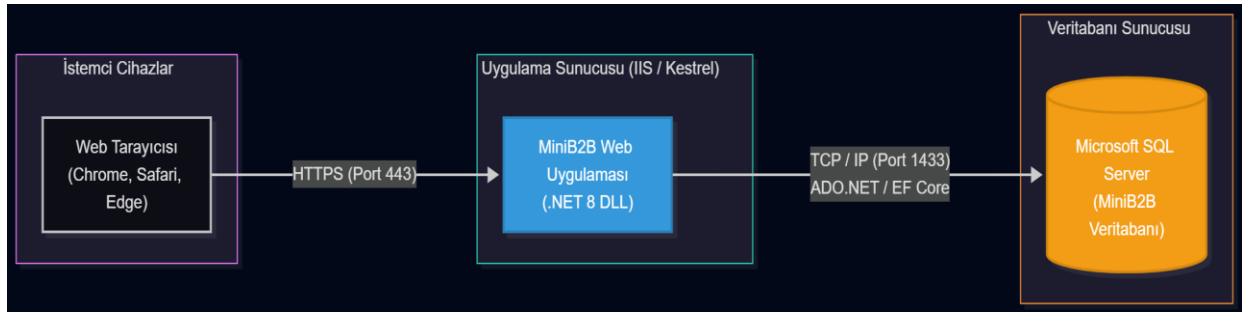


Şekil 6. Sequence Diagramı

7. Dağıtım Diyagramı (Deployment Diagram)

Projenin canlı ortamda (Production) nasıl barındırılacağını ve sunucu mimarisini özetler:

- **İstemci (Client):** Kullanıcılar Chrome, Edge veya Safari gibi tarayıcılar üzerinden sisteme bağlanır. Güvenlik için aradaki tüm iletişim standart **HTTPS (Port 443)** ile şifrelenir.
- **Uygulama Sunucusu:** .NET 8 üzerinde derlenen projenin çalıştığı Kestrel / IIS ortamıdır. Tüm Business Logic ve MVC motoru bu sunucu üzerinde koşar.
- **Veritabanı Sunucusu:** SQL Server ayrı bir sunucu üzerinde izole şekilde tutulur. Uygulama sunucusu veritabanına doğrudan **TCP/IP (Port 1433)** üzerinden güvenli connection string ile bağlanır. Bu ayrım hem güvenlik hem de sunucu yükünü bölmek açısından tercih edilmiştir.



Şekil 7. Dağıtım Diyagramı (Deployment Diagram)